

zkGCN: Zero-Knowledge Proofs of Inference for Graph Convolutional Networks *

Hongfa Ding^{1,2*}, Linjiang Wu¹, Yingxuan Luo¹, Heling Jiang¹

¹Information School, Guizhou University of Finance and Economics, Guiyang, 550025, Guizhou, China.

²Guizhou Zhonghui Technology Development Co., Ltd, Guiyang, 550000, Guiyang, China.

*Corresponding author(s). E-mail(s): hfding@mail.gufe.edu.cn;

Abstract

In recent years, Graph Neural Networks (GNN) (e.g. Graph Convolutional Networks, GCN) as an emerging learning model, have shown outstanding performance in tasks involving graph data, and deployed in many machine learning and application platforms. Although their applications have achieved some success, current GNN MLaaS platforms lack verifiable capability to ensure the integrity of the inference procedure. Consequently, users cannot confirm whether the inference results they received are correct or not, nor can they confirm whether these results are generated by the specific model they paid for or not. To address these issues, we propose zkGCN, the first zero-knowledge proof-based integrity verification scheme for the GCN inference procedure. This scheme can verify the integrity of the GCN inference procedure in GNN MLaaS scenarios without exposing the private parameter information, thereby ensuring that the inference results are the output of a legitimate model. In this scheme, zk-SNARK is employed to construct zero-knowledge proofs of the GCN inference process, and we construct rank-1 constraint satisfaction based on the computational logic of each hidden layer in the inference procedure of the model, and then design proof circuits. To validate the effectiveness of our approach, we performed graph classification experiments with GCN models over three benchmark datasets, demonstrating both the practicality and the feasibility of the proposed scheme.

Keywords: Graph Neural Network, Graph Convolutional Networks, Machine Learning as a Service, Zero-Knowledge, zk-SNARK, Verifiable Inference.

*This study was funded by the Natural Science Researching Program of D. o. E. of Guizhou (grant number: Qian Edu. & Tech.[2023]065) and the Innovation Exploration and Academic New Seed Project of GUFU (grant number: 2024XSXMB11).

1 Introduction

With the rapid development of artificial intelligence and big data analytics technologies, GNN (e.g. GCN) as an emerging deep learning model has demonstrated excellent performance in tasks involving graph structured data. GNN models have also gradually become commercial MLaaS. Many MLaaS platforms provided by cloud service providers have integrated GCN development tools to facilitate the launch of GCN services. Users only need to utilize the application programming interfaces (APIs) provided by the cloud platform and pay relevant fees to directly access pre-trained GCN models. Although model deployment is convenient and cost-effective, the integrity[1, 2] of the graph learning-based MLaaS inference procedure lacks verifiable capabilities. Consequently, users cannot confirm whether the inference results they receive originate from the specific model they paid for or whether the inference results of the paid model are accurate. Users still do not know the internal structure, algorithms, parameters, and other confidential details of the GCN model hosted on the MLaaS platform when using inference services. Dishonest providers might provide substandard services, such as providing predictions from simplified models to defraud users. To verify whether the GCN model provided by the cloud service provider offers genuine services, the model owner needs to disclose internal privacy parameters. However, this method risks exposing key model parameters. Therefore, it is crucial to provide integrity assurance for inference services without leaking model privacy parameters. Certain protection schemes for graph learning models [3–5] can detect integrity attacks and safeguard model intellectual property to enhance security, but do not meet the requirements of MLaaS scenarios where both cloud service providers and clients demand balanced trust, privacy and verifiable integrity[6, 7].

To this end, we propose zkGCN, the first Zero-Knowledge Proof (ZKP)-based scheme to verify the integrity of the GCN inference procedure. Without exposing the private parameter information of the GCN model, our scheme verifies that the inference procedure strictly adheres to the given input, operates under the specified model architecture, and follows the predefined computational procedures, thereby ensuring the trustworthiness of the final inference results. This indirectly demonstrates the credibility of both the inference outcomes and the model providing the service. ZKP guarantees that if the prover sends an incorrect computation result, it can only be correctly verified with negligible probability. The cloud provider computes the model inference procedure and uses zk-SNARK to construct a ZKP of the model inference procedure without revealing private information such as model parameters during the integrity proof. Furthermore, the client can use the proof to verify the integrity of the inferred result of the model. Verification of the ZKP system will fail if the model parameters are changed or the model structure is modified. The main contributions of this paper are as follows.

1. We propose zkGCN, the first zk-SNARK-based scheme to verify the integrity of the GCN inference procedure. This scheme strategically adopts the zero-knowledge property of zk-SNARK and the protection of model private parameters, which can smoothly solve the integrity verification of the model and prevent the leakage of model parameters.

2. We propose an algorithm for extracting the computational logic of the internal layers of the GCN model and converting them into the Rank-1 Constraint Satisfaction (R1CS) required by the ZKP circuit.

3. We implement and comprehensively evaluate zkGCN over the GCN model with different public datasets. Experimental results show that the proposed scheme has a relatively manageable time overhead and an attractive advantage in terms of proof size.

2 Related Work

In this section, we will review and analyze related work on verifiable DNN and GNN. These works include integrity verification schemes for both DNN inference and DNN training phases and privacy-preserving GNN schemes.

2.1 Integrity Verification of DNN

To address fraudulent services and privacy concerns in MLaaS, recent research has explored methods to verify the integrity and ownership of DNN models deployed on MLaaS platforms during the inference phase. He et al. [8] propose a protection mechanism for DNN models in MLaaS based on sensitive sample fingerprints. It requires more sophisticated fingerprint computation algorithms to generate sensitive samples and cannot fully guarantee the detection of all potential model integrity breaches. VCNN[9] is a verifiable convolutional neural network framework based on zk-SNARKs. The computational resources required to generate proofs are still high, especially when dealing with large-scale models. ZkCNN[10] can predict the accuracy of CNN models while protecting the privacy of the model parameters. However, it relies on integer quantization, which can affect model accuracy and increase implementation complexity.

There is also research exploring the integrity verification of the training procedure of DNNs. Kasra et al. proposed a zkPoT system named KAIZEN[11]. KAIZEN achieves integrity verification of the DNN training procedure by combining optimized proofs for gradient descent with recursive composition techniques. VeriML[12] leverages SNARKs on randomly selected iterations of the ML training procedure to achieve efficiency and tunable probabilistic guarantees. However, it lacks strong assurances in areas such as service correctness, trustworthy billing, and fair payment. zkDL[13] introduces zkReLU, which designs specialized proofs for the ReLU activation function and its backpropagation in deep neural networks. It also introduces a novel arithmetic circuit design called FAC4DNN to simulate the entire training procedure.

2.2 GNN Models Privacy-Preserving

Some works focus on the privacy issues during the GNN model training procedure. Iyiola [14] proposes PrivGnn, which is a privacy-preserving framework for publishing GNN models with differential privacy guarantees. Xu[5] introduces a privacy-preserving GNN named MDP, which allows external calculators to train GNN models without knowing the original data. SecGNN[15] is a system that combines

additive secret sharing with machine learning techniques to design encrypted data structures and secure protocols, enabling privacy-preserving GNN training and inference services in the cloud. Wu et al. [3] proposed a query-based verification technique, complemented by an innovative node fingerprint generation algorithm, introducing a random fingerprint node method to protect GNN models in MLaaS from model-centric attacks, thus safeguarding the integrity of the models.

3 Preliminaries

In this section, to make the proposed zkGCN easier to understand, we briefly introduce some basic concepts.

3.1 Graph Convolutional Networks

The GCN procedure can be formalized as follows. Given a graph $G = (V, E)$ with node features $H \in \mathbb{R}^{N \times D}$, where N is the number of nodes and D is the number of features per node, a graph convolution operation can be defined as

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} + b), \quad (1)$$

where $H^{(l)}$ is the feature matrix at layer l , $W^{(l)}$ is the learnable weight matrix at layer l , $\tilde{A} = A + I_N$ is the adjacency matrix A with self-loops added (where I_N is the identity matrix), $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$, b is an optional bias term. σ is a non-linear activation function, typically ReLU.

3.2 zk-SNARKs

zk-SNARK [16] enables efficient verification of computational integrity with three key properties: **Zero-knowledge**, No information about the witness $\mathbf{w}$ is revealed. **Succinctness**, Constant-sized proofs. **Non-interactivity**, Single-round verification requiring only public inputs. For a computational statement A with secret witness $\mathbf{w}$ and public input x , the protocol operates as follows:

1. **Arithmetic Circuit and R1CS**: Decompose the computation into basic arithmetic operations (e.g., $f(x) = x^3 + x + 5$). Represent these as a R1CS: $\mathbf{s} \cdot \mathbf{q}_i = (\mathbf{s} \cdot \mathbf{o}_i) \cdot (\mathbf{s} \cdot \mathbf{d}_i)$, where $\mathbf{s} = (1, x, \mathbf{w})$ is the full variable vector, and $\mathbf{o}_i, \mathbf{d}_i, \mathbf{q}_i$ are coefficient matrices.
2. **QAP Transformation**: Convert R1CS to Quadratic Arithmetic Program (QAP) by Lagrange interpolation. Let $p(x) = q(x) - o(x) \cdot d(x)$. Verification reduces to checking: $p(x) = h(x) \cdot z(x)$, where $z(x) = \prod_{i=1}^m (x - x_i)$ is the vanishing polynomial.
3. **Groth16 Protocol**[17]: **Setup**, Generate keys $(PK, VK) \leftarrow G(C, 1^\lambda)$. **Prove**, Compute proof $\pi \leftarrow P(PK, \mathbf{w}, x)$. **Verify**, Check $V(VK, x, \pi) \in \{0, 1\}$. G denotes the setup phase of the proof system for generating public parameters PK and VK , and the P, V represent the provers and verifiers. The P and V engage in a non-interactive protocol, where the prover convinces the verifier of the validity of a statement without revealing the actual witness w .

4 The Proposed zkGCN Scheme

In this section, we first present the overview of our zkGCN scheme, and then present the specific details of zkGCN.

4.1 Overview of zkGCN

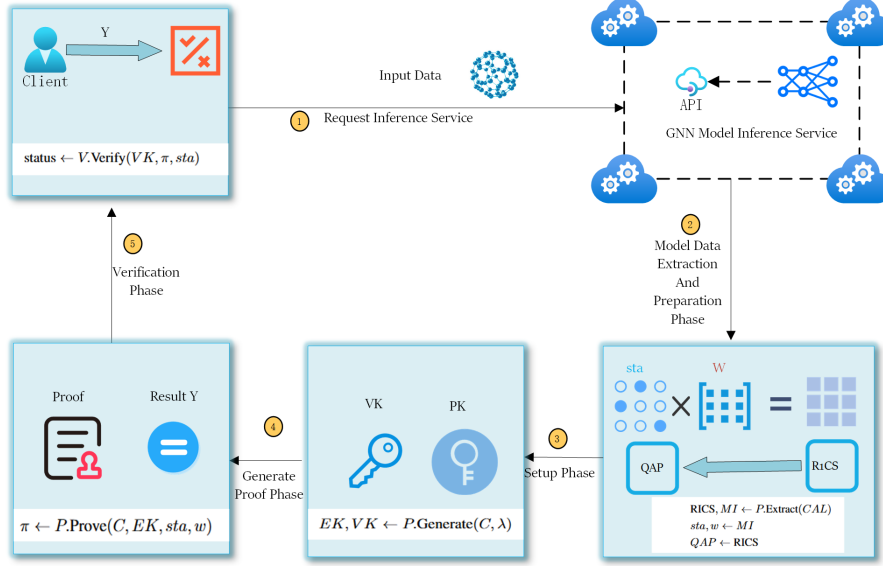


Fig. 1: Overview of the zkGCN.

In the GCN MLaaS scenario, the prover (cloud service provider) has a pre-trained GCN model that remains confidential. The verifier (user) requests inference services from the prover with some input data. The prover needs to demonstrate to the verifier (user) that a complete inference process was executed using this underlying GCN model on the user's input data, thereby yielding an authentic and correct inferred result. To meet this requirement, we assume that the model architecture is known to the verifier and propose the zkGCN scheme to provide integrity verification of the GCN inference procedure. To the best of our knowledge, this is the first such ZKP-based scheme for GCN inference. The workflow of zkGCN is illustrated in Fig. 1.

In Fig. 1, there are four core components and five steps. 1) After publishing the GCN model M by the prover, the verifier queries the inference service for a specific data sample D . 2) The prover extracts matrix information MI (includes public primary input sta and the witness w , where, sta represents the public input, including the input and output of the first layer, the input of the second layer, etc., while w denotes the

private parameters, such as model weights, biases, and other confidential components.) from each layer’s computation logic CAL in the GCN inference process, translates CAL into simple arithmetic expression and arithmetic circuits C , constructs R1CS based on C , and finally converts R1CS into an arithmetic QAP polynomial. 3) The prover generates keys PK (for proof generation) and VK (for verification) based on the circuit size. 4) The verifier generates an integrity proof π using C , sta , w , and PK based on the Gorth16 system and sends it to the verifier. If it is the final layer, the inferred result Y is also sent to the verifier. 5) The verifier verifies the integrity of all layer using PK and π . Once all the layers are verified, the verifier considers Y to be the legitimate output of the model.

4.2 Details of zkGCN

4.2.1 Preparation for the Construction of R1CS

We extracted the information about computation procedure of each layer in the GCN model. Then, the computational logic of the corresponding layers is converted into simple arithmetic expressions (SAE). and then transform them into R1CS. NP-complete statement R1CS can be used to describe the logical computation of polynomial problems, preparing for the implementation of the zk-SNARK proof scheme.

Due to the lack of support for floating-point arithmetic operations in ZKP’s backend, it is necessary to handle floating-point numbers that arise during the model inference process, we use the fixed-point decimal representation from Ref.[18]. We represent the floating-point numbers in the computation information with an integer-type fraction matrix. Specifically, we select an integer k as a correction parameter, multiply each element of the original floating-point matrix by 2^k to convert it into an integer, forming the numerator matrix. The elements in the denominator matrix are all coefficients 2^k . This conversion process introduces a potential loss of precision in the original floating-point matrix, with the accuracy range of the loss being $[0, 2^{k-1}]$. By selecting an appropriate k , the accuracy loss can be controlled within a range that does not affect validation.

GCN Convolutional Layer: As can be seen from Eq. 1, by performing symmetric normalization on the adjacency matrix $\hat{A} = (\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}})$, the normalized adjacency matrix is multiplied with the current layer’s feature matrix H^l , aggregating the feature information of neighboring nodes to obtain $Conv_Input_Norm$. To verify the integrity of the GCN convolution layer, we use the normalized and aggregated node feature data $Conv_Input_Norm$, the weight matrix $Conv_weight$, the bias term $Conv_bias$, and the output of the convolution function $Conv_output$ to represent the logical calculation of the convolutional layer. It can be expressed by the following formula:

$$Conv_Input_Norm \cdot Conv_weight + Conv_bias = Conv_output \quad (2)$$

We use Alg. 1 to transform the computational procedure of the convolutional layer (matrix multiplication) into SAE and further construct the R1CS.

GCN Activation Function Layer: We take the Rectified Linear Unit (ReLU) as an example, transforming the computational logic of the activation layer into a

Algorithm 1 GCNCONV-layer conversion Matrix Multiplication to R1CS

Input: Params: GCNCONV-layer(H^l , $weight$, $bias$, $output$)

Transform: $conv_input_Norm = \hat{A} \cdot H^l$

Output: R1CS

```
1: {conv_input_Norm.csv, conv_weight.csv, conv_bias.csv, conv_output.csv} ←  
   Params  
2: {conv_input_Norm[n][m], conv_weight[m][p], conv_bias[p], conv_output[m][p]} ←  
   Read *.csv  
3: for i = 0 to n - 1, j = 0 to p - 1  
4:   Construct_SAE(i,j)  
5:   RICS ← SAEij  
6: end for  
7: function CONSTRUCT_SAE(i, j)  
8:   Initialize a two-dimensional array  $Sym[n \times p][m + 1]$   
9:   sum_row = 0  
10:  for m = 0 to m - 1  
11:     $Sym[p * i + j][m] \leftarrow conv\_input[i][m] * conv\_weight[j][m]$   
12:    sum_row ← sum_row +  $Sym[p * i + j][m]$   
13:  end for  
14:   $Sym[p * i + j][m] \leftarrow sum\_row + conv\_bias[j]$   
15:  Flatten GCNCONV-layer Matrix multiplication  
16:  return SAEij  
17: end function
```

simple arithmetic expression. The upper section of Fig. 2 details the simple arithmetic expression of the ReLU activation layer. ReLU can be represented $f(x) = \max(0, x)$, where x is the output value from the previous layer. To express the computational logic of the *ReLU* activation layer, we construct an *ReLU_operation* matrix, the shape is same input matrix (*ReLU_input*). When the elements of the input matrix to the activation layer are greater than zero, the corresponding values in the operation matrix are set to 1, and to 0 otherwise. Finally, by performing the Hadamard product between the *ReLU_input* matrix and the operation matrix, we obtain the output of the *ReLU* layer (*ReLU_output*). In this way, the ReLU function is transformed into a form similar to matrix multiplication. This can be represented by the formula as

$$ReLU_input \circ ReLU_operation = ReLU_output. \quad (3)$$

GCN Global Average Pooling Layer: To transform the computational logic of global mean pooling into matrix operations, we multiply a unit vector *GMP_operation* of dimension equal to the number of nodes with the *GMP_input* matrix. The resulting *GMP_output* represents exactly the computational logic of the global pooling operation. The construction of a simple arithmetic expression is analogous to Alg. 1, except it is formulated as the multiplication of a matrix and a vector. This can be represented

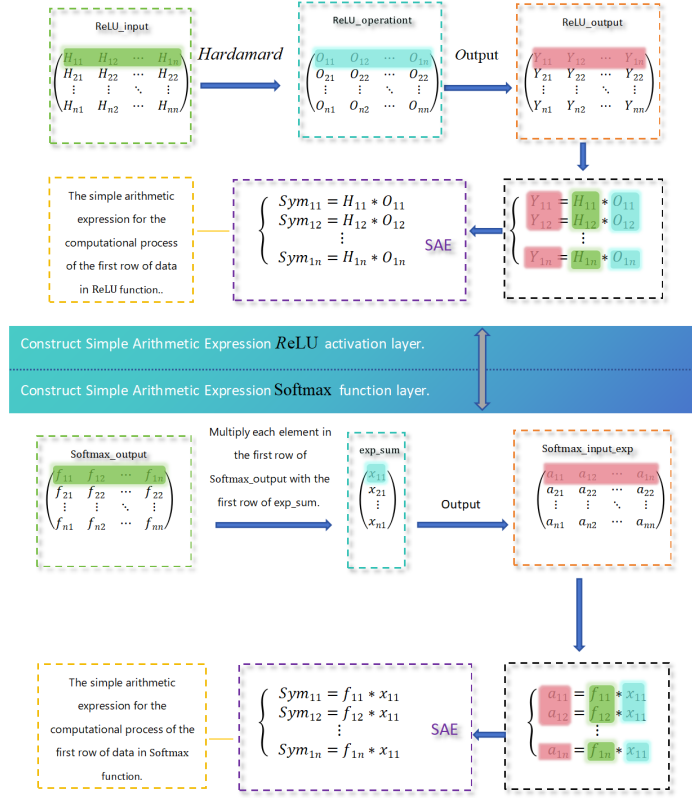


Fig. 2: Simple arithmetic expressions for the ReLU layer and the Softmax layer.

by the formula as follows:

$$GMP_operation \cdot GMP_input = GMP_output \quad (4)$$

GCN Fully Connected layer: In our model, the Fully-Connected Layer serves as a "classifier". Typically, one or more Fully-Connected Layers follow the graph convolutional layers to perform task-related transformations and predictions on the nodes' high-order structural representations. Specifically, the computation in each fully connected layer is as follows:

$$Fc_input \cdot Fc_Weight + Fc_bais = Fc_output \quad (5)$$

where Fc_input represents the output matrix of the nodes after the ReLU layer, with each row being a node's feature vector, Fc_Weight is the weight matrix, and Fc_bais is the bias vector. We extract the Fc_input , Fc_Weight , Fc_bais , and Fc_output data

from the fully connected layer and construct our simple arithmetic expression based on the upper part of Fig. 2, preparing for the R1CS.

4.3 The zk-SNARK Process of zkGCN

The zk-SNARK process include the reminding three components of zk-GCN, namely setup, proof generation, and verification phases.

1. **Setup phase:** The computational logic to be verified is first compiled into arithmetic circuits and R1CS, and then converted into QAP polynomials $A(t)B(t) - C(t) = 0$. Generate public parameters CRS including PK, VK using the coefficients $A_i(t)$ and $B_i(t)$ of the QAP polynomial. It can be expressed as $(ParrowG(C, 1^{lambda}))$. Grepresents a function that generates common parameters PK and VK .
2. **Proof generation phase:** Based on PK and input data, the prover calculates polynomials $P(t)$ and $h(t)$, where $h(t) = P(t)/z(t)$, $z(t) = \prod_{k=1}^d (t - \tau_k)$, τ_k are randomly sampled points, d is the Polynomial degree. The prover encrypts $P(s)$ and $h(s)$ using the parameters in PK and generates a proof π . Can be represented as $\pi \leftarrow P(PK, w, sta)$, where w is witness and sta is public input.
3. **Verification phase:** The verifier uses a public reference string (VK) and proof π to verify the correctness of the calculation. Firstly, the verifier receives public input, proof π and VK , and then calculates the encrypted form of the objective polynomials $t(s)$ and $P(s)$. Finally, Verify that a following bilinear pairing equations hold. Can be represented as $V(VK, sta, \pi) \in \{0, 1\}$. Please refer to zk-snarks[16] for the specific implementation process.

5 Theoretical Analysis

We will prove the security of zkGCN, and compare zkGCN with other related schemes theoretically in this section.

5.1 Security Analysis

The security of zkGCN can be proved based on the three fundamental properties (i.e. completeness, soundness, and zero-knowledge) of ZKP, and as Theorem 1.

Theorem 1. *If the three properties are satisfied, the proposed zkGCN is considered to fulfill the properties of a ZKP.*

Proof Completeness. For any input from the verifier (user), if the prover correctly extracts the relevant matrix data information of the internal computational logic at each layer during the GCN inference, and generates an integrity proof using the arithmetic circuit and the public security parameter PK , then the verifier will always obtain a verification result of 1 via VK , π , and the public input.

$$\begin{cases} \Pr[(PK, VK) \leftarrow \text{GCNlayer. KeyGen}(C, 1^\lambda) \\ \pi \leftarrow \text{Prove}(PK, w, sta) \\ \text{Verify}(VK, sta, \pi) = 1] = 1 \end{cases} \quad (6)$$

Soundness. For any probabilistic polynomial-time (PPT) malicious adversary algorithm, the probability

$$\Pr \left[\begin{array}{l} (PK, VK) \leftarrow \text{GCNlayer. KeyGen}(1^\lambda) \\ \pi^* \leftarrow \mathcal{A}(C, PK) \\ V. \text{Verify}(sta^*, \pi^*, VK) = 1 \\ \exists sta \in MI, sta^* \neq sta \text{ } sta^* \in MI^* \end{array} \right] \quad (7)$$

holds is negligible, where MI is the matrix data extracted at the layer, sta is the public primary input and $\mathcal{A}$ denotes the adversary.

Zero-knowledge. Let there exist simulators S_1, S_2 , and PPT algorithms $\mathcal{A}_1, \mathcal{A}_2$. Here, pp is the security parameter, sta denotes public input, w denotes private input (witness), and GCNlayer represents a layer in the GCN model. Consider the two experiments in Table 1, a scheme is said to satisfy the zero-knowledge property if the difference in output probabilities between the real experiment ($\text{EXP}_{\text{real}}(pp)$) and the simulated experiment ($\text{EXP}_{\text{simulator}}(pp)$) is negligible. Formally, this is

$$|\Pr[\text{EXP}_{\text{real}}(pp) = 1] - \Pr[\text{EXP}_{\text{simulator}}(pp) = 1]| \leq \text{negl}(\lambda). \quad (8)$$

□

Table 1: Comparison of Different Experiments

$\text{EXP}_{\text{real}}(pp)$	$\text{EXP}_{\text{simulator}}(pp)$
1) $pp \leftarrow \text{GCNlayer. Generate}(C, 1^\lambda)$	1) $pp \leftarrow S_1(C, 1^\lambda)$
2) $(s, w) \leftarrow \mathcal{A}_1(\text{GCNlayer})$	2) $(s, w) \leftarrow \mathcal{A}_1(\text{GCNlayer})$
3) $\pi \leftarrow P(pp, s, w)$	3) $\pi \leftarrow S_2(pp, s)$
4) $b \leftarrow \mathcal{A}_2(pp, s, w, \pi)$	4) $b \leftarrow \mathcal{A}_2(pp, s, w, \pi)$
5) <i>Return</i> b	5) <i>Return</i> b

5.2 Functionality Comparison and Analysis

To illustrate the properties of zkGCN, we compare some related schemes in Table 2. In Table 2, we can see that zkGCN is the first ZKP-based integrity verification scheme for GCN inference. Although there are integrity verification schemes [9, 10, 19, 20], they just work for the traditional ML training (i.e. CNN, linear and logistic regression) [19] or NN inference [9, 10, 20]. Wu et al. [3] employ a fingerprint algorithm to verify whether the GNN model is modified by the cloud service provider. This approach only probabilistically verifies integrity and cannot guarantee the privacy of the GNN model. In our zkGCN scheme, the zkSNARK protocol is used to ensure the privacy of the model and the integrity of the inference process. The completeness and correctness of the derived results can be verified once by the prover and verifier, respectively.

Table 2: Comparison of Different Schemes of Verifiable Machine Learning

Scheme	Phase of ML	Method	Verifiability	Privacy
zkMLaaS[19]	ML training	zk-SNARK	○	●
Mystique[20]	NN inference	MPC-in-the-Head-based	●	●
vCNN[9]	CNN inference	commit and prove SNARK,	●	●
zkCNN[10]	CNN inference	Sumcheck and commit-and-prove SNARK	●	●
Wu et al [3]	GNN inference	Node Fingerprint Algorithm	◐	◐
zkGCN(Ours)	GCN inference	zk-SNARK	●	●

○denotes the requirement is not satisfied; ◐denotes the requirement is partially satisfied; ●denotes the requirement is fully satisfied;

6 Experimental Implementation and Evaluation

6.1 Experiment Settings

We selected three graph classification datasets, namely MUTAG, NCI1, and Reddit-Binary, and the details of them are shown in Table 3. We conducted experiments using a single graph as a case study to predict its classification. We implemented the Groth16 protocol using libsnark[21] to generate and verify integrity proofs for each layer of the model. All experiments were run on a Thinkpaid laptop with an Intel(R) Core(TM) i5-8250U CPU and 16 GB of RAM.

Table 3: Datasets Overview

Dataset	Graphs	Classes	Average Nodes	Node Features
MUTAG	188	2	17.9	7
RDDIT-BINARY	2000	2	429.61	0
NCI1	4110	2	29.8	37

6.2 Evaluation and Comparison

This section evaluates the time and storage overhead of zkGCN by inferring individual graph samples. The time cost of zkGCN includes the setup, proof generation, and verification stages, while the storage cost includes the size of PK , VK and GCN model. We also compare these overheads with those of vCNN[9] and zkCNN[10].

6.2.1 Evaluation

Time Overhead: As shown in Fig. 3, the time overhead of zkGCN on the three datasets. Since the inference phase of GCN models for individual graph data is very short, we ignore it here. Due to the relatively small average number of nodes per graph (17.9) on the MUTAG dataset, the circuit design remains simple, resulting in lower computational overhead. Specifically, the total time cost during the setup phase is approximately 5.2 seconds, while the proof generation phase requires 4.1 seconds in

total. The verification phase completes within 878 ms. On NCI1, the setup phase takes about 9.4 seconds, the proof generation phase takes about 8 seconds, and the validation phase takes about 1 second. On REDDIT-BINARY, the setup phase takes about 40 seconds, the proof generation phase takes about 38 seconds, and the validation phase takes about 1.1 seconds. Due to the average number of nodes per graph in NCI1 being 429.6, the time cost of zkGCN is slightly higher than in NCI and MUTAG, but it is acceptable for practical situations.

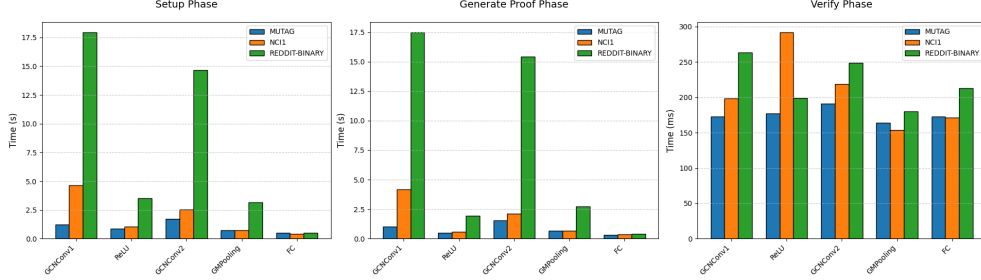


Fig. 3: Time costs of three stages of zkGCN on different graph datasets.

Storage Overhead: Table 4 shows the storage overhead of the GCN model integrity verification circuit on three datasets. This includes proving the parameter PK and validating the parameter VK , as well as the size of the model on different datasets. We represent the size of the model by using the model parameters, assuming that each parameter occupies 4 bytes. For lightweight datasets such as MUTAG and NCI1, the total storage costs for PK and VK are 2.26 MB and 5.78 MB, respectively. These costs increase linearly with the matrix dimension of the GCN layer, as the number of arithmetic operations directly affects the complexity of the R1CS circuit. For larger graph classification datasets (REDDIT-BINARY), the overhead will also increase with the increase of computational complexity. From a practical application perspective, REDDIT-BINARY is a relatively large graph classification dataset, and these storage requirements are completely feasible and acceptable. For users, they only need to bear the cost of verifying the key, which is relatively small, proving the feasibility of the integrity verification scheme based on zk-SNARK in implementing graph classification tasks.

6.2.2 Comparison

Both vCNN[9] and zkCNN[10] use similar methods with our zkGCN, but zkGCN's targets and data structures differ, rendering a comparison of overheads not substantially meaningful. We evaluate the feasibility of our scheme based on the time spent by the prover, the verifier, and the parameter storage overhead(CRS). We show the differences of these in Table 5. As shown in Table 5, for vCNN, the time spent by the prover reaches 18 hours and the storage overhead reaches 83 GB for large models (VGG16), which is unacceptable in reality, thus showing relative limitations. For

Table 4: The Storage Consumptions of zkGCN over Different Datasets

Dataset	Layer	PK	VK	Model Size	Total
MUTAG	GCNConv1	511 KB	20 KB	3.38 KB	2.26 MB
	ReLU	149 KB	28 KB		
	GCNConv2	1.11 MB	318 KB		
	GMPooling	125 KB	14 KB		
	FC	13 KB	1 KB		
NCI1	GCNConv1	3.37 MB	376 KB	3.57 KB	5.78 MB
	ReLU	193 KB	35 KB		
	GCNConv2	1.40 MB	286 KB		
	GMPooling	164 KB	19 KB		
	FC	13 KB	1 KB		
REDDIT-BINARY	GCNConv1	15.42 MB	376 KB	20.13 KB	35.85 MB
	ReLU	1.98 MB	376 KB		
	GCNConv2	15.42 MB	376 KB		
	GMPooling	1.73 MB	189 KB		
	FC	13 KB	1 KB		

zkCNN, acceptable time overhead is demonstrated even for large models, but the storage overhead is not stated in the paper, making it immeasurable. For zkGCN, which targets graph classification tasks, we increase the scale of model integrity verification by enlarging the dataset size. In our experiments, for relatively large graph classification datasets, both time and storage overheads remain acceptable in practice.

Table 5: Experimental comparison between other schemes

vCNN			
Model	Prover	Verifier	CRS
LeNet-5	28.81 s	75 ms	40.07 MB
AlexNet	38 min	130 ms	2.1 GB
VGG16	18 hours	19.4 s	83 GB
zkCNN			
Model	Prover	Verifier	CRS
LeNet-5	0.441 s	5.80 ms	N/A
VGGG11	47.8 s	39.3 ms	N/A
VGGG16	88.3 s	59.3 ms	N/A
zkGCN			
Dataset	Prover	Verifier	CRS
MUTAG	9.3 s	878 ms	2.26 MB
NCI1	17.4 s	1.0 s	5.78 MB
REDDIT-BINARY	78 s	1.1 s	35.85 MB

7 Conclusion

This paper proposes a ZKP-based integrity verification scheme named zkGCN for GCN inference. zkGCN pioneers strict verification of the GCN inference process without leaking model parameters, bridging the technological gap in existing solutions for complex graph data scenarios. Experimental results demonstrate that zkGCN achieves manageable time and storage overheads in lightweight graph classification. In the future, more general and efficient ZKP-based verification methods for other GNN models and inference tasks should be explored.

References

- [1] Liang, W., Fan, Y., Li, K., Zhang, D., Gaudiot, J.: Sbbs: A secure blockchain-based scheme for iot data credibility in fog environment. *IEEE Internet of Things Journal* **8**(11), 9268–9277 (2021)
- [2] Liang, W., Fan, Y., Li, K., Zhang, D., Gaudiot, J.: Secure data storage and recovery in industrial blockchain network environments. *IEEE Transactions on Industrial Informatics* **16**(10), 6543–6552 (2020)
- [3] Wu, B., Yuan, X., Wang, S.: Securing graph neural networks in MLaaS: A comprehensive realization of query-based integrity verification. In: 2024 IEEE Symposium on Security and Privacy, SP 2024, pp. 2534–2552. IEEE
- [4] Dai, E., Lin, M., Wang, S.: Pregip: Watermarking the pretraining of graph neural networks for deep intellectual property protection. *arXiv preprint arXiv:2402.04435* (2024)
- [5] Xu, W., Shi, B., Zhang, J., Feng, Z., Pan, T., Dong, B.: Mdp: Privacy-preserving gnn based on matrix decomposition and differential privacy. In: 2023 IEEE International Conference on Joint Cloud Computing, pp. 38–45. IEEE
- [6] Fan, Y., Lin, X., Liang, W.: A secure privacy preserving deduplication scheme for cloud computing. *Future Generation Computer Systems* **101**, 127–135 (2019)
- [7] Fan, Y., Liu, J., Li, K.-C., Lei, X., Tan, G., Tang, M., *et al.*: One enhanced secure access scheme for outsourced data. *Information sciences* **561**, 230–242 (2021)
- [8] He, Z., Zhang, T., Lee, R.: Sensitive-sample fingerprinting of deep neural networks. In: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, pp. 4729–4737. IEEE
- [9] Lee, S., Ko, H., Kim, J., Oh, H.: vCNN: Verifiable convolutional neural network based on zk-snarks. *IEEE Transactions on Dependable and Secure Computing* **21**(4), 4254–4270 (2024)
- [10] Liu, T., Xie, X., Zhang, Y.: zkCNN: Zero knowledge proofs for convolutional

- neural network predictions and accuracy. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS 2021, pp. 2968–2985. ACM, New York, USA (2021)
- [11] Abbaszadeh, K., Pappas, C., Katz, J., Papadopoulos, D.: Zero-knowledge proofs of training for deep neural networks. In: 2024 ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, pp. 4316–4330. ACM
 - [12] Zhao, L., Wang, Q., Wang, C., Li, Q., Shen, C., Feng, B.: Veriml: Enabling integrity assurances and fair payments for machine learning as a service. *IEEE Transactions on Parallel and Distributed Systems* **32**(10), 2524–2540 (2021)
 - [13] Sun, H., Bai, T., Li, J., Zhang, H.: zkDL: Efficient zero-knowledge proofs of deep learning training. *IEEE Transactions on Information Forensics and Security* **20**, 914–927 (2025)
 - [14] Wang, B., Guo, J., Li, A., Chen, Y., Li, H.: Privacy-preserving representation learning on graphs: A mutual information perspective. In: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2021, pp. 1667–1676. ACM
 - [15] Wang, S., Zheng, Y., Jia, X.: SecGNN: Privacy-preserving graph neural network training and inference as a cloud service. *IEEE Transactions on Services Computing* **16**(4), 2923–2938 (2023)
 - [16] Petkus, M.: Why and how zk-snark works. CoRR **abs/1906.07221** (2019) [1906.07221](https://arxiv.org/abs/1906.07221)
 - [17] Groth, J.: On the size of pairing-based non-interactive arguments. In: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT 2016, pp. 305–326. Springer
 - [18] Zhao, L., Wang, Q., Wang, C., Li, Q., Shen, C., Feng, B.: VeriML: Enabling integrity assurances and fair payments for machine learning as a service. *IEEE Transactions on Parallel and Distributed Systems* **32**(10), 2524–2540 (2021)
 - [19] Huang, C., Wang, J., Chen, H., Si, S., Huang, Z., Xiao, J.: zkMLaaS: a verifiable scheme for machine learning as a service. In: IEEE Global Communications Conference, GLOBECOM 2022, pp. 5475–5480. IEEE
 - [20] Parthiban, E.S., Adil, M.: Seeing through the mystique surrounding the task-technology-organization fit: A mixed-method exploration of consumers’ intention to use banking conversational agents. *Communications of the Association for Information Systems* **55**, 17 (2024)
 - [21] scipr-lab: libsnark: C++ library for zkSNARKs. <https://github.com/scipr-lab/libsnark>. Accessed: 2025-03-05 (2014)